

Guía Completa de Threads en Java para Junior Developers

Tabla de Contenidos

1. Introducción: ¿Qué son los Threads?
 2. Concepto 1: Thread.sleep() - La Pausa Básica
 3. Concepto 2: wait() y la Sincronización
 4. Concepto 3: Monitores y Locks
 5. Concepto 4: Multithreading Real
 6. Concepto 5: Interrupciones
 7. Concepto 6: wait() vs sleep() - La Gran Diferencia
 8. Concepto 7: Patrones Avanzados
 9. Mejores Prácticas y Errores Comunes
 10. Ejercicios Prácticos
-

Introducción: ¿Qué son los Threads?

Definición Simple

Un **thread** (hilo) es como un trabajador independiente dentro de tu programa. Imagina una oficina donde cada empleado (thread) puede trabajar en su propia tarea al mismo tiempo que los demás.

Analogía de la Oficina

Programa = Oficina completa
Process = Departamento
Thread = Empleado individual
Recurso compartido = Impresora de la oficina
Monitor/Lock = Llave de la sala de reuniones

Tu Primer Thread en Java

```
java
```

```
public class MiPrimerThread {
    public static void main(String[] args) {
        System.out.println("Hilo principal: " + Thread.currentThread().getName());

        // Crear un nuevo thread
        Thread nuevoThread = new Thread(() -> {
            System.out.println("Nuevo hilo: " + Thread.currentThread().getName());
        });

        nuevoThread.start(); // ¡NO uses run()! Usa start()
    }
}
```

🔑 **Punto Clave:** `start()` crea un nuevo thread, `run()` ejecuta en el thread actual.

Concepto 1: Thread.sleep() - La Pausa Básica

¿Qué hace Thread.sleep()?

Pausa el thread actual por un tiempo específico **SIN liberar ningún lock/monitor**.

Analogía de la Pizza

Imagina que estás en una pizzería con una sola mesa:

- **Thread.sleep()** = Te duermes EN la mesa con tu pizza
- Nadie más puede sentarse aunque estés dormido
- La mesa sigue "ocupada" por ti

Código Ejemplo - V0 Mejorado

```
java
```

```

package com.tutorial.sleep;

public class ThreadSleepDemo {

    // Recurso compartido
    private static final Object mesa = new Object();

    public static void main(String[] args) {
        System.out.println("=== Demo Thread.sleep() ===\n");

        // Thread 1: Cliente que se duerme en la mesa
        Thread cliente1 = new Thread(() -> {
            synchronized(mesa) {
                System.out.println(timestamp() + "Cliente 1: Ocupé la mesa");
                System.out.println(timestamp() + "Cliente 1: Me voy a dormir 5 seg...");
                try {
                    Thread.sleep(5000); // Duerme PERO no suelta el lock
                    System.out.println(timestamp() + "Cliente 1: Desperté, sigo en la mesa");
                } catch (InterruptedException e) {
                    System.out.println(timestamp() + "Cliente 1: Me despertaron!");
                }
                System.out.println(timestamp() + "Cliente 1: Me voy, mesa libre");
            }
        });

        // Thread 2: Cliente esperando la mesa
        Thread cliente2 = new Thread(() -> {
            System.out.println(timestamp() + "Cliente 2: Quiero la mesa...");
            synchronized(mesa) {
                System.out.println(timestamp() + "Cliente 2: ¡Por fin conseguí la mesa!");
            }
        });

        cliente1.start();
        try { Thread.sleep(100); } catch (Exception e) {} // Asegurar orden
        cliente2.start();
    }

    private static String timestamp() {
        return "[" + (System.currentTimeMillis() % 100000) / 1000.0 + "s] ";
    }
}

```

Output Esperado:

```
Cliente 1: Ocupé la mesa
Cliente 1: Me voy a dormir 5 seg...
Cliente 2: Quiero la mesa...    // Bloqueado esperando
[5 segundos después]
Cliente 1: Desperté, sigo en la mesa
Cliente 1: Me voy, mesa libre
Cliente 2: ¡Por fin conseguí la mesa!
```

Problemas con Thread.sleep():

1. **No libera locks** - Bloquea recursos innecesariamente
 2. **No es cooperativo** - Otros threads no pueden usar el recurso
 3. **Desperdicia recursos** - El recurso está inactivo pero bloqueado
-

Concepto 2: wait() y la Sincronización

El Error Clásico (V1)

```
java

package com.tutorial.wait.error;

public class WaitSinSyncError {
    public static void main(String[] args) {
        Object obj = new Object();

        try {
            obj.wait(); // ❌ ERROR: IllegalMonitorStateException
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Analogía del Baño Público

- **wait() sin synchronized** = Intentar esperar dentro del baño sin tener la llave
- El guardia (JVM) te detiene: "¡No puedes esperar ahí sin permiso!"

✓ La Forma Correcta (V2)

```
java

package com.tutorial.wait.correcto;

public class WaitConSync {

    private static final Object bano = new Object();

    public static void main(String[] args) throws InterruptedException {
        System.out.println("=== Demo wait() Correcto ===\n");

        Thread persona1 = new Thread(() -> {
            synchronized(bano) {
                System.out.println("Persona 1: Entré al baño (tengo la llave)");
                System.out.println("Persona 1: Voy a esperar 3 seg y SUELTO la llave");
                try {
                    bano.wait(3000); // Espera Y libera el monitor
                    System.out.println("Persona 1: Volví y recuperé la llave");
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        Thread persona2 = new Thread(() -> {
            try { Thread.sleep(500); } catch (Exception e) {} // Dejar que P1 entre primero

            synchronized(bano) {
                System.out.println("Persona 2: ¡Pude entrar mientras P1 esperaba!");
                System.out.println("Persona 2: Salgo rápido");
            }
        });

        persona1.start();
        persona2.start();

        persona1.join();
        persona2.join();
    }
}
```



Diferencia Clave:

java

// Thread.sleep() - Egoísta

```
synchronized(recurso) {  
    Thread.sleep(5000); // Mantiene el lock 🔒  
}
```

// wait() - Cooperativo

```
synchronized(recurso) {  
    recurso.wait(5000); // Libera el lock 🔓  
}
```



Concepto 3: Monitores y Locks



¿Qué es un Monitor?

Un monitor es como una "llave maestra" que controla el acceso a un objeto. Solo un thread puede tener esta llave a la vez.



Analogía del Banco

java

```
package com.tutorial.monitors;

public class BancoConMonitor {

    private double saldo = 1000;
    private final Object cajaFuerte = new Object(); // Monitor externo

    // Método 1: Monitor del propio objeto (this)
    public synchronized void retirarForma1(double cantidad) {
        System.out.println(Thread.currentThread().getName() +
            " tiene la llave del banco (this)");
        if (saldo >= cantidad) {
            saldo -= cantidad;
            System.out.println("Retiro exitoso. Saldo: " + saldo);
        }
    }

    // Método 2: Monitor de objeto externo (más flexible)
    public void retirarForma2(double cantidad) {
        synchronized(cajaFuerte) { // Usa monitor específico
            System.out.println(Thread.currentThread().getName() +
                " tiene la llave de la caja fuerte");
            if (saldo >= cantidad) {
                saldo -= cantidad;
                System.out.println("Retiro exitoso. Saldo: " + saldo);
            }
        }
    }

    // Método 3: Combinando wait() para coordinación
    public void esperarSaldo(double cantidadMinima) {
        synchronized(cajaFuerte) {
            while (saldo < cantidadMinima) {
                System.out.println("Esperando saldo suficiente...");
                try {
                    cajaFuerte.wait(); // Libera la caja fuerte
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            System.out.println("¡Hay saldo suficiente!");
        }
    }
}
```

```
public void depositar(double cantidad) {  
    synchronized(cajaFuerte) {  
        saldo += cantidad;  
        System.out.println("Depósito: +" + cantidad + ". Saldo: " + saldo);  
        cajaFuerte.notifyAll(); // Despierta a los que esperan  
    }  
}  
}
```

🎓 Tipos de Sincronización:

1 Sincronización de Método (Monitor: this)

```
java  
  
public synchronized void metodo() {  
    // Todo el método está protegido por 'this'  
}
```

2 Sincronización de Bloque (Monitor: objeto específico)

```
java  
  
public void metodo() {  
    // Código no sincronizado  
    synchronized(objetoMonitor) {  
        // Solo este bloque está protegido  
    }  
    // Más código no sincronizado  
}
```

3 Sincronización Estática (Monitor: Class object)

```
java  
  
public static synchronized void metodoEstatico() {  
    // Usa MyClass.class como monitor  
}
```

Concepto 4: Multithreading Real

Múltiples Threads Compitiendo

java

```
package com.tutorial.multithreading;

import java.util.concurrent.atomic.AtomicInteger;

public class CarreraDeThreads {

    private static final Object pista = new Object();
    private static AtomicInteger posicion = new AtomicInteger(1);
    private static final int TOTAL_CORREDORES = 3;

    static class Corredor implements Runnable {
        private final String nombre;
        private final int tiempoCarrera;

        Corredor(String nombre, int tiempoCarrera) {
            this.nombre = nombre;
            this.tiempoCarrera = tiempoCarrera;
        }

        @Override
        public void run() {
            System.out.println(nombre + " comenzó a correr!");

            // Simular carrera
            try {
                Thread.sleep(tiempoCarrera);
            } catch (InterruptedException e) {
                System.out.println(nombre + " fue interrumpido!");
                return;
            }

            // Llegar a la meta (sincronizado)
            synchronized(pista) {
                int miPosicion = posicion.getAndIncrement();
                System.out.println(nombre + " llegó en posición #" + miPosicion);

                // Si no soy el último, espero a los demás
                if (miPosicion < TOTAL_CORREDORES) {
                    try {
                        System.out.println(nombre + " espera a los demás...");
                        pista.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
        }
    }
}
```

```

    }
} else {
    // Soy el último, despierto a todos
    System.out.println(nombre + " fue el último. ¡Todos a celebrar!");
    pista.notifyAll();
}

    System.out.println(nombre + " está celebrando 🎉");
}
}
}

public static void main(String[] args) throws InterruptedException {
    Thread corredor1 = new Thread(new Corredor("🏃 Juan", 2000));
    Thread corredor2 = new Thread(new Corredor("🏃 María", 1500));
    Thread corredor3 = new Thread(new Corredor("🏃 Pedro", 2500));

    System.out.println("🏁 ¡Comienza la carrera!\n");

    corredor1.start();
    corredor2.start();
    corredor3.start();

    corredor1.join();
    corredor2.join();
    corredor3.join();

    System.out.println("\n🏆 ¡Carrera terminada!");
}
}

```

🎮 Patrón Productor-Consumidor

java

```

package com.tutorial.patterns;

import java.util.LinkedList;
import java.util.Queue;

public class ProductorConsumidor {

    private final Queue<Integer> buffer = new LinkedList<>();
    private final int CAPACIDAD = 5;
    private final Object lock = new Object();

    class Productor implements Runnable {
        @Override
        public void run() {
            int valor = 0;
            while (true) {
                synchronized(lock) {
                    while (buffer.size() == CAPACIDAD) {
                        try {
                            System.out.println("📦 Buffer lleno, productor espera...");
                            lock.wait();
                        } catch (InterruptedException e) {
                            Thread.currentThread().interrupt();
                            return;
                        }
                    }

                    buffer.offer(valor);
                    System.out.println("➕ Producido: " + valor +
                        " | Buffer: " + buffer.size());
                    valor++;

                    lock.notifyAll(); // Despertar consumidores

                    try {
                        Thread.sleep(500);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            }
        }
    }
}

```

```
}
```

```
class Consumidor implements Runnable {
    @Override
    public void run() {
        while (true) {
            synchronized(lock) {
                while (buffer.isEmpty()) {
                    try {
                        System.out.println("🚗 Buffer vacío, consumidor espera...");
                        lock.wait();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            }

            int valor = buffer.poll();
            System.out.println("🚗 Consumido: " + valor +
                               " | Buffer: " + buffer.size());

            lock.notifyAll(); // Despertar productores

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}

public static void main(String[] args) {
    ProductorConsumidor pc = new ProductorConsumidor();

    Thread productor = new Thread(pc.new Productor(), "Productor");
    Thread consumidor1 = new Thread(pc.new Consumidor(), "Consumidor-1");
    Thread consumidor2 = new Thread(pc.new Consumidor(), "Consumidor-2");

    productor.start();
    consumidor1.start();
    consumidor2.start();
}
```

Concepto 5: Interrupciones

¿Qué es interrupt()?

Es una forma educada de pedirle a un thread que termine lo que está haciendo.

Analogía del Semáforo

- Thread normal = Auto en la carretera
- `interrupt()` = Luz amarilla intermitente
- El auto DECIDE si detenerse o continuar

java

```
package com.tutorial.Interrupciones;
```

```
public class InterrupcionesDemo {
```

```
    static class TareaInterrumpible implements Runnable {
```

```
        @Override
```

```
        public void run() {
```

```
            try {
```

```
                System.out.println("✍ Comenzando tarea larga...");
```

```
                for (int i = 1; i <= 10; i++) {
```

```
                    // Verificar si nos han interrumpido
```

```
                    if (Thread.currentThread().isInterrupted()) {
```

```
                        System.out.println("🛑 Me pidieron parar educadamente");
```

```
                        return;
```

```
                    }
```

```
                    System.out.println("Trabajando... " + i + "/" + 10);
```

```
                    Thread.sleep(1000);
```

```
                }
```

```
                System.out.println("✅ Tarea completada");
```

```
            } catch (InterruptedException e) {
```

```
                System.out.println("⚠ Fui interrumpido mientras dormía/esperaba");
```

```
                // Restaurar el estado de interrupción
```

```
                Thread.currentThread().interrupt();
```

```
            }
```

```
        }
```

```
    }
```

```
    static class TareaNoCooperativa implements Runnable {
```

```
        @Override
```

```
        public void run() {
```

```
            System.out.println("😡 Soy rebelde, ignoro interrupciones");
```

```
            while (true) {
```

```
                // No chequeo isInterrupted()
```

```
                // No uso sleep/wait
```

```
                // ¡Nunca pararé! (MAL DISEÑO)
```

```
                System.out.print(".");
```

```
                // Simulación de trabajo intensivo
```

```

        for (int i = 0; i < 1000000; i++) {
            Math.sqrt(i);
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    // Ejemplo 1: Thread cooperativo
    System.out.println("=== Thread Cooperativo ===");
    Thread trabajador1 = new Thread(new TareaInterrumpible());
    trabajador1.start();

    Thread.sleep(3500);
    System.out.println("Main: Interrumpiendo trabajador cooperativo...");
    trabajador1.interrupt();

    trabajador1.join();

    // Ejemplo 2: Thread no cooperativo (comentado porque no termina)
    /*
    System.out.println("\n=== Thread No Cooperativo ===");
    Thread trabajador2 = new Thread(new TareaNoCooperativa());
    trabajador2.start();

    Thread.sleep(2000);
    System.out.println("Main: Interrumpiendo trabajador rebelde...");
    trabajador2.interrupt(); // No servirá de nada
    */
}
}

```

Reglas de Oro para Interrupciones:

1. **Siempre** verifica `isInterrupted()` en loops largos
2. **Siempre** maneja `InterruptedException` apropiadamente
3. **Considera** restaurar el flag con `Thread.currentThread().interrupt()`
4. **Nunca** ignores una interrupción sin buena razón

 Concepto 6: wait() vs sleep() - La Gran Diferencia

 Tabla Comparativa Completa

Aspecto	Thread.sleep()	Object.wait()
Libera monitor	✗ NO	✓ Sí
Requiere synchronized	✗ NO	✓ Sí
Se puede despertar con notify()	✗ NO	✓ Sí
Método estático/instancia	Estático	Instancia
Propósito	Pausa simple	Coordinación

 Demostración Práctica

```
java
```

```
package com.tutorial.comparison;

public class WaitVsSleepDemo {

    private static final Object recurso = new Object();

    static class UsandoSleep implements Runnable {
        @Override
        public void run() {
            synchronized(recurso) {
                System.out.println(Thread.currentThread().getName() +
                    " - sleep: Tengo el recurso");
                try {
                    System.out.println(" Durmiendo 3 seg (NO suelto el lock)...");
                    Thread.sleep(3000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println(" Desperté y SIGO con el lock");
            }
        }
    }

    static class UsandoWait implements Runnable {
        @Override
        public void run() {
            synchronized(recurso) {
                System.out.println(Thread.currentThread().getName() +
                    " - wait: Tengo el recurso");
                try {
                    System.out.println(" Esperando 3 seg (Sí suelto el lock)...");
                    recurso.wait(3000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println(" Volví y recuperaré el lock");
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("=== Escenario 1: Con SLEEP ===");
        long inicioSleep = System.currentTimeMillis();
```

```
Thread t1 = new Thread(new UsandoSleep(), "Thread-1");
Thread t2 = new Thread(new UsandoSleep(), "Thread-2");

t1.start();
t2.start();

t1.join();
t2.join();

long tiempoSleep = System.currentTimeMillis() - inicioSleep;
System.out.println("Tiempo total con sleep: " + tiempoSleep + "ms");
System.out.println("(Nota: ~6 segundos porque es SECUENCIAL)\n");

System.out.println("=== Escenario 2: Con WAIT ===");
long inicioWait = System.currentTimeMillis();

Thread t3 = new Thread(new UsandoWait(), "Thread-3");
Thread t4 = new Thread(new UsandoWait(), "Thread-4");

t3.start();
t4.start();

t3.join();
t4.join();

long tiempoWait = System.currentTimeMillis() - inicioWait;
System.out.println("Tiempo total con wait: " + tiempoWait + "ms");
System.out.println("(Nota: ~3 segundos porque pueden SOLAPARSE)");
}
}
```

Concepto 7: Patrones Avanzados

Patrón: wait() + sleep() Combinados

java

```
package com.tutorial.advanced;

public class PatronHibrido {

    private static final Object token = new Object();

    static class TareaHibrida implements Runnable {
        private final String nombre;
        private final int fase;

        TareaHibrida(String nombre, int fase) {
            this.nombre = nombre;
            this.fase = fase;
        }

        @Override
        public void run() {
            synchronized(token) {
                // Fase 1: Espera cooperativa
                System.out.println(nombre + " - Fase " + fase +
                    ": Esperando turno (wait)...");

                try {
                    token.wait(2000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                // Fase 2: Trabajo exclusivo
                System.out.println(nombre + " - Fase " + fase +
                    ": Trabajo crítico (sleep)...");

                try {
                    Thread.sleep(1000); // Mantiene el lock
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                System.out.println(nombre + " - Fase " + fase +
                    ": Completado");
            }
        }
    }

    public static void main(String[] args) {
```

```
for (int i = 1; i <= 3; i++) {  
    new Thread(new TareaHibrida("Worker", i)).start();  
}  
}  
}
```

Patrón: notify() vs notifyAll()

java

```
package com.tutorial.advanced;

public class NotifyVsNotifyAll {

    private static final Object sala = new Object();
    private static boolean recursoListo = false;

    static class Esperador implements Runnable {
        private final String nombre;

        Esperador(String nombre) {
            this.nombre = nombre;
        }

        @Override
        public void run() {
            synchronized(sala) {
                while (!recursoListo) {
                    System.out.println(nombre + " esperando...");
                    try {
                        sala.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                System.out.println(nombre + " ¡Recurso obtenido!");
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        // Crear múltiples esperadores
        for (int i = 1; i <= 3; i++) {
            new Thread(new Esperador("Esperador-" + i)).start();
        }

        Thread.sleep(2000);

        synchronized(sala) {
            System.out.println("\n🔔 Usando notify() - Solo uno despierta:");
            sala.notify(); // Solo despierta a UNO
        }
    }
}
```

```
Thread.sleep(2000);
```

```
synchronized(sala) {  
    System.out.println("\n🔔🔔 Usando notifyAll() - Todos despiertan:");  
    recursoListo = true;  
    sala.notifyAll(); // Despierta a TODOS  
}  
}  
}
```

✅ Mejores Prácticas y Errores Comunes

🚫 Errores Comunes

❌ Error 1: Olvidar synchronized con wait()

```
java  
  
// MAL  
object.wait(); // IllegalMonitorStateException  
  
// BIEN  
synchronized(object) {  
    object.wait();  
}
```

❌ Error 2: Sincronizar en objetos mutables

```
java  
  
// MAL  
private Integer contador = 0;  
synchronized(contador) { // ¡contador puede cambiar!  
    contador++;  
}  
  
// BIEN  
private final Object lock = new Object();  
synchronized(lock) {  
    contador++;  
}
```

❌ Error 3: No manejar InterruptedException

```
java

// MAL
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Ignorar - MAL
}

// BIEN
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // Restaurar estado
    return; // Salir limpiamente
}
```

✅ Mejores Prácticas

1. Usa java.util.concurrent cuando sea posible

```
java

// En lugar de wait/notify manual
BlockingQueue<String> queue = new LinkedBlockingQueue<>();

// Productor
queue.put("item");

// Consumidor
String item = queue.take(); // Espera automáticamente si está vacía
```

2. Siempre usa try-finally con locks explícitos

```
java
```

```
private final ReentrantLock lock = new ReentrantLock();

public void metodoSeguro() {
    lock.lock();
    try {
        // Código crítico
    } finally {
        lock.unlock(); // Siempre se ejecuta
    }
}
```

3. Evita sincronización anidada (previene deadlocks)

```
java

// PELIGROSO
synchronized(lock1) {
    synchronized(lock2) { // Posible deadlock
        // ...
    }
}

// MEJOR: Adquirir locks en orden consistente
```

Ejercicios Prácticos

Ejercicio 1: Semáforo Simple

Implementa un semáforo que permita máximo 3 threads simultáneos:

```
java
```

```
public class Semaforo {  
    private int permisos = 3;  
  
    public synchronized void adquirir() {  
        // TODO: Esperar si no hay permisos  
    }  
  
    public synchronized void liberar() {  
        // TODO: Liberar un permiso y notificar  
    }  
}
```

Ejercicio 2: Barrera de Sincronización

Crea una barrera que espere a que N threads lleguen antes de continuar:

```
java  
  
public class Barrera {  
    private final int totalThreads;  
    private int threadEsperando = 0;  
  
    public Barrera(int n) {  
        this.totalThreads = n;  
    }  
  
    public synchronized void esperar() {  
        // TODO: Implementar lógica de barrera  
    }  
}
```

Ejercicio 3: Pool de Conexiones

Diseña un pool simple de conexiones con wait/notify:

```
java
```

```
public class ConnectionPool {
    private final Queue<Connection> pool;
    private final int maxSize;

    public synchronized Connection obtenerConexion() {
        // TODO: Esperar si no hay conexiones disponibles
    }

    public synchronized void devolverConexion(Connection conn) {
        // TODO: Devolver conexión y notificar esperadores
    }
}
```

Ejercicio 4: Sistema de Turnos

Implementa un sistema donde los threads se ejecutan en orden específico:

```
java

public class SistemaDeTurnos {
    private int turnoActual = 1;

    public synchronized void esperarTurno(int miTurno) {
        // TODO: Esperar hasta que sea mi turno
    }

    public synchronized void siguienteTurno() {
        // TODO: Avanzar al siguiente turno
    }
}
```



Soluciones a los Ejercicios

Solución Ejercicio 1: Semáforo Simple

```
java
```

```

public class Semaforo {
    private int permisos = 3;

    public synchronized void adquirir() throws InterruptedException {
        while (permisos == 0) {
            System.out.println(Thread.currentThread().getName() +
                               " esperando permiso...");
            wait();
        }
        permisos--;
        System.out.println(Thread.currentThread().getName() +
                           " adquirió permiso. Disponibles: " + permisos);
    }

    public synchronized void liberar() {
        permisos++;
        System.out.println(Thread.currentThread().getName() +
                           " liberó permiso. Disponibles: " + permisos);
        notify(); // Despertar a un esperador
    }

    // Test
    public static void main(String[] args) {
        Semaforo semaforo = new Semaforo();

        for (int i = 1; i <= 5; i++) {
            new Thread(() -> {
                try {
                    semaforo.adquirir();
                    Thread.sleep(2000); // Simular trabajo
                    semaforo.liberar();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }, "Thread-" + i).start();
        }
    }
}

```

Solución Ejercicio 2: Barrera de Sincronización

java

```
public class Barrera {
    private final int totalThreads;
    private int threadsEsperando = 0;
    private boolean barrepaRota = false;

    public Barrera(int n) {
        this.totalThreads = n;
    }

    public synchronized void esperar() throws InterruptedException {
        if (barrepaRota) {
            throw new IllegalStateException("Barrera ya fue usada");
        }

        threadsEsperando++;

        if (threadsEsperando == totalThreads) {
            // Último thread llegó, liberar a todos
            System.out.println("¡Todos llegaron! Liberando barrera...");
            barrepaRota = true;
            notifyAll();
        } else {
            // Esperar a que lleguen los demás
            System.out.println(Thread.currentThread().getName() +
                               " esperando (" + threadsEsperando +
                               "/" + totalThreads + ")");
            while (threadsEsperando < totalThreads && !barrepaRota) {
                wait();
            }
        }

        System.out.println(Thread.currentThread().getName() +
                           " pasó la barrera!");
    }

    // Test
    public static void main(String[] args) {
        Barrera barrera = new Barrera(3);

        for (int i = 1; i <= 3; i++) {
            final int id = i;
            new Thread(() -> {
                try {
```

```
        Thread.sleep(id * 1000); // Llegan en diferentes momentos
        System.out.println("Thread-" + id + " llegó a la barrera");
        barrera.esperar();
        System.out.println("Thread-" + id + " continúa trabajando");
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}).start();
}
}
```

Solución Ejercicio 3: Pool de Conexiones

```
java
```

```
import java.util.*;

// Simulación de conexión
class Connection {
    private final int id;
    private boolean enUso = false;

    Connection(int id) {
        this.id = id;
    }

    public void usar() {
        System.out.println("Usando conexión " + id);
    }

    public int getId() {
        return id;
    }
}

public class ConnectionPool {
    private final Queue<Connection> disponibles = new LinkedList<>();
    private final Set<Connection> enUso = new HashSet<>();
    private final int maxSize;

    public ConnectionPool(int maxSize) {
        this.maxSize = maxSize;
        // Crear conexiones iniciales
        for (int i = 1; i <= maxSize; i++) {
            disponibles.offer(new Connection(i));
        }
    }

    public synchronized Connection obtenerConexion() throws InterruptedException {
        while (disponibles.isEmpty()) {
            System.out.println(Thread.currentThread().getName() +
                               " esperando conexión...");
            wait();
        }

        Connection conn = disponibles.poll();
        enUso.add(conn);
        System.out.println(Thread.currentThread().getName() +
```

```

        " obtuvo conexión " + conn.getId());
    return conn;
}

public synchronized void devolverConexion(Connection conn) {
    if (!enUso.contains(conn)) {
        throw new IllegalArgumentException("Conexión no pertenece al pool");
    }

    enUso.remove(conn);
    disponibles.offer(conn);
    System.out.println(Thread.currentThread().getName() +
        " devolvió conexión " + conn.getId());
    notify(); // Despertar a un esperador
}

// Test
public static void main(String[] args) {
    ConnectionPool pool = new ConnectionPool(2); // Solo 2 conexiones

    // Crear 5 threads que necesitan conexiones
    for (int i = 1; i <= 5; i++) {
        new Thread(() -> {
            try {
                Connection conn = pool.obtenerConexion();
                conn.usar();
                Thread.sleep(2000); // Simular uso
                pool.devolverConexion(conn);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }, "Cliente-" + i).start();
    }
}
}

```

Solución Ejercicio 4: Sistema de Turnos

```
java
```

```

public class SistemaDeTurnos {
    private int turnoActual = 1;

    public synchronized void esperarTurno(int miTurno) throws InterruptedException {
        while (turnoActual != miTurno) {
            System.out.println("Thread con turno " + miTurno +
                               " esperando (turno actual: " + turnoActual + ")");
            wait();
        }
        System.out.println("¡Es el turno " + miTurno + "!");
    }

    public synchronized void siguienteTurno() {
        turnoActual++;
        System.out.println("Avanzando a turno " + turnoActual);
        notifyAll(); // Despertar a todos para que verifiquen
    }

    // Test: Threads ejecutándose en orden específico
    public static void main(String[] args) {
        SistemaDeTurnos sistema = new SistemaDeTurnos();

        // Crear threads en orden aleatorio
        int[] ordenCreacion = {3, 1, 4, 2, 5};

        for (int turno : ordenCreacion) {
            new Thread(() -> {
                try {
                    sistema.esperarTurno(turno);
                    System.out.println("Thread con turno " + turno +
                                       " ejecutando trabajo...");
                    Thread.sleep(1000);
                    sistema.siguienteTurno();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}

```

1. Sistema de Cache con Expiración

java

```

public class CacheConExpiracion<K, V> {
    private final Map<K, V> cache = new HashMap<>();
    private final Map<K, Long> tiemposExpiracion = new HashMap<>();
    private final long tiempoVida;

    public CacheConExpiracion(long tiempoVidaMs) {
        this.tiempoVida = tiempoVidaMs;

        // Thread limpiador
        Thread limpiador = new Thread(() -> {
            while (true) {
                synchronized (this) {
                    long ahora = System.currentTimeMillis();
                    Iterator<Map.Entry<K, Long>> it =
                        tiemposExpiracion.entrySet().iterator();

                    while (it.hasNext()) {
                        Map.Entry<K, Long> entry = it.next();
                        if (entry.getValue() < ahora) {
                            cache.remove(entry.getKey());
                            it.remove();
                            System.out.println("Cache: Eliminado " +
                                entry.getKey());
                        }
                    }
                }
            }

            try {
                Thread.sleep(1000); // Verificar cada segundo
            } catch (InterruptedException e) {
                break;
            }
        });
        limpiador.setDaemon(true);
        limpiador.start();
    }

    public synchronized void put(K key, V value) {
        cache.put(key, value);
        tiemposExpiracion.put(key, System.currentTimeMillis() + tiempoVida);
        notify(); // Por si alguien espera este valor
    }
}

```

```

public synchronized V get(K key) throws InterruptedException {
    while (!cache.containsKey(key)) {
        wait(tiempoVida); // Esperar hasta timeout
        if (!cache.containsKey(key)) {
            return null; // No llegó a tiempo
        }
    }
    return cache.get(key);
}
}

```

2. Sistema de Mensajería Simple

```

java

public class SistemaMensajeria {
    private final Map<String, Queue<String>> buzonesUsuarios = new HashMap<>();

    public synchronized void enviarMensaje(String usuario, String mensaje) {
        buzonesUsuarios.computeIfAbsent(usuario, k -> new LinkedList<>())
            .offer(mensaje);
        notifyAll(); // Notificar a todos los que esperan mensajes
    }

    public synchronized String recibirMensaje(String usuario)
        throws InterruptedException {
        Queue<String> buzón = buzonesUsuarios.get(usuario);

        while (buzón == null || buzón.isEmpty()) {
            System.out.println(usuario + " esperando mensajes...");
            wait(); // Esperar nuevo mensaje
            buzón = buzonesUsuarios.get(usuario);
        }

        return buzón.poll();
    }
}

```

Próximos Pasos

Temas Avanzados para Explorar:

1. java.util.concurrent

- `ExecutorService` y Thread Pools
- `CompletableFuture` para programación asíncrona
- `CountDownLatch`, `CyclicBarrier`, `Phaser`

2. Locks Avanzados

- `ReentrantLock` con fairness
- `ReadWriteLock` para lecturas concurrentes
- `StampedLock` para optimización

3. Estructuras Thread-Safe

- `ConcurrentHashMap`
- `BlockingQueue` implementations
- `CopyOnWriteArrayList`

4. Atomic Operations

- `AtomicInteger`, `AtomicReference`
- Compare-and-swap (CAS)
- Lock-free programming

Recursos Recomendados:

1. Libros:

- "Java Concurrency in Practice" - Brian Goetz
- "Effective Java" (Capítulo de Concurrencia) - Joshua Bloch

2. Documentación:

- [Oracle Java Tutorials - Concurrency](#)
- [Java API Documentation](#)

3. Práctica:

- LeetCode Concurrency Problems
 - HackerRank Threading Challenges
-

💡 Consejos Finales

🎯 Reglas de Oro para Junior Developers:

1. **Comienza Simple:** No sobre-complices con threads si no es necesario
2. **Prefiere Alto Nivel:** Usa `java.util.concurrent` antes que wait/notify
3. **Evita Estado Compartido:** Minimiza la necesidad de sincronización
4. **Prueba Exhaustivamente:** Los bugs de concurrencia son difíciles de reproducir
5. **Documenta:** Explica POR QUÉ usas sincronización, no solo cómo

⚠️ Señales de Advertencia:

- Si necesitas más de 2 niveles de sincronización anidada
- Si tu código tiene muchos wait/notify manuales
- Si es difícil explicar el flujo de ejecución
- Si los tests fallan intermitentemente

Probablemente necesitas repensar el diseño.

🎉 ¡Felicitaciones!

Has completado la guía completa de Threads en Java. Ahora tienes las bases para:

- Entender código concurrente existente
- Identificar y corregir problemas de sincronización
- Diseñar soluciones thread-safe básicas
- Continuar aprendiendo conceptos más avanzados

¡La práctica hace al maestro! 💪